

Neural Network Foundations

DAY 1 COMPLETE | Built and interpreted a single artificial neuron from scratch

Learning outcome

Understand how inputs, weights, bias, activation functions, and thresholds work together during a forward pass, and connect this process to logistic regression from Week 3.

What you demonstrated

- Calculated weighted sums manually and with NumPy.
- Implemented sigmoid without TensorFlow or scikit-learn.
- Applied one neuron to multiple customer samples.
- Compared binary predictions at thresholds 0.50 and 0.65.
- Separated samples, feature inputs, hidden layers, and output neurons.
- Explained why manual weight editing is not model training.

1. The artificial neuron

An artificial neuron is a mathematical unit. It receives input features, multiplies each feature by a weight, adds the results and a bias, and passes the weighted sum through an activation function.

Inputs -> Weighted sum + bias -> Activation -> Output

Core equations

Weighted sum: $z = x_1w_1 + x_2w_2 + \dots + x_nw_n + b$

Activated output: $a = f(z)$

Symbol	Meaning	Role
x	Input feature	A measured value supplied to the neuron
w	Weight	Controls the feature's influence on z
b	Bias	Shifts the weighted sum and decision boundary
z	Pre-activation	Weighted sum before activation
a	Activation output	Value passed forward or used as output

2. Connection to Week 3 machine learning

Model	Calculation	Interpretation
Linear regression	Weighted sum	A continuous numeric prediction
Logistic regression	Weighted sum + sigmoid	A binary-class probability
Single sigmoid neuron	Weighted sum + sigmoid	Mathematically equivalent binary output
Neural network	Many connected neurons	Learns nonlinear representations through layers

Logistic regression can be represented as one sigmoid output neuron with no hidden layers.

Weights and bias

A positive weight pushes z upward when its input increases. A negative weight pushes z downward. The bias is a learned parameter that shifts z . During prediction, the bias is already known; it is not calculated from the current customer's features.

Worked churn example

Feature	Input	Weight	Contribution
Tenure	0.20	-1.50	-0.30
Monthly Charges	0.80	1.20	0.96
Support Calls	0.70	0.80	0.56
Bias	-	-	-0.20

$$z = -0.30 + 0.96 + 0.56 - 0.20 = 1.02$$

$$\text{sigmoid}(1.02) = 0.7350$$

At threshold 0.50: $0.7350 \geq 0.50$, so predicted class = 1.

3. Activation functions

Function	Definition / behavior	Common use
ReLU	$\max(0, z)$	Hidden layers
Sigmoid	$1 / (1 + \exp(-z))$	Binary-classification output
Softmax	Converts logits into probabilities summing to 1	Multiclass output, such as MNIST

Why activation matters: several linear layers without nonlinear activations can be simplified into one linear transformation. Nonlinear activations let networks learn more complex relationships.

4. Layers and architecture

Layer	Purpose
Input	Provides feature values; it is not a hidden layer.
Hidden	Learns intermediate representations between input and output.
Output	Produces the final value, probability, or class probabilities.

Current architecture: 3 feature inputs -> 0 hidden layers -> 1 sigmoid output neuron

Samples are not neurons

The four customers are four data samples. Each sample supplies three feature values. The same output neuron processes every sample using the same weights and bias. Four customers do not create four neurons.

5. Forward propagation and training

Forward propagation uses the current parameters to produce predictions. It does not update them.

Forward pass -> Loss -> Backpropagation -> Optimizer update

Stage	What happens
Forward pass	The network produces predictions.
Loss	A function measures prediction error.
Backpropagation	Gradients are calculated for weights and biases.
Optimizer	Parameters are updated using the gradients.

Parameters versus hyperparameters

Type	Examples	Determined by
Parameters	Weights, biases	Learned during training
Hyperparameters	Learning rate, epochs, layer size	Chosen before or during experimentation

6. Decision thresholds

Sigmoid produces a probability. The threshold converts that probability into a class. Changing the threshold does not change the probability; it changes only the classification rule.

Probability	Threshold	Predicted class
0.5793	0.50	1
0.5793	0.65	0

7. Clean NumPy implementation

```
import numpy as np

weights = np.array([-1.50, 1.20, 0.80])
bias = -0.20

customers = {
    "Customer A": np.array([0.10, 0.90, 0.80]),
    "Customer B": np.array([0.85, 0.40, 0.10]),
    "Customer C": np.array([0.40, 0.60, 0.50]),
    "Customer D": np.array([0.70, 0.75, 0.20]),
}

def sigmoid(value):
    return 1 / (1 + np.exp(-value))

def predict_churn(customer_features, weights, bias, threshold):
    weighted_sum = np.dot(customer_features, weights) + bias
    probability = sigmoid(weighted_sum)
    predicted_class = int(probability >= threshold)
    return weighted_sum, probability, predicted_class

for threshold in (0.50, 0.65):
    print(f"Threshold: {threshold:.2f}")
    for name, features in customers.items():
        z, probability, prediction = predict_churn(
            features, weights, bias, threshold
        )
        print(name, f"z={z:.4f}",
              f"p={probability:.4f}", f"class={prediction}")
```

Use the same predict_churn function for every threshold. Redefining the function for threshold 0.65 is unnecessary because threshold is already an argument.

Mini-project results

Customer	z	Probability	Class @ 0.50	Class @ 0.65
A	1.3700	0.7974	1	1
B	-0.9150	0.2860	0	0
C	0.3200	0.5793	1	0
D	-0.1900	0.4526	0	0

Interpretation

- Customer A has the highest churn probability; Customer B has the lowest.
- Customer C changes from class 1 to class 0 when the threshold rises to 0.65.
- Customer C's tenure contribution is -0.6000, so tenure pushes z downward.
- Customer C's monthly-charges contribution is +0.7200, so it pushes z upward.

8. Common mistakes and corrections

Mistake	Correction
Calling an output neuron a hidden layer	The current model has zero hidden layers and one output neuron.
Treating four samples as four neurons	The same neuron processes four separate customer samples.
Saying backpropagation updates parameters	Backpropagation calculates gradients; the optimizer updates parameters.
Rewriting a function for a new threshold	Reuse one function and pass a different threshold argument.
Reading 1.019999999999998 as an error	It is normal floating-point representation; display it as 1.0200.
Calling manual weight editing training	Training uses loss, gradients, and an optimizer to learn parameters.

Five-minute revision sheet

Question	Quick answer
What does a neuron calculate?	Weighted sum + bias, followed by activation.
What controls feature influence?	Weights.
What shifts the decision boundary?	Bias.
Typical hidden activation?	ReLU.
Typical binary output activation?	Sigmoid.
Typical MNIST output activation?	Softmax over 10 outputs.
Does a forward pass update weights?	No. It produces predictions.
What does backpropagation produce?	Gradients.
What updates parameters?	The optimizer.
Current architecture?	3 inputs -> 0 hidden layers -> 1 sigmoid output.

Day 1 takeaway: a neural network grows from the same weighted-sum and sigmoid idea already used in logistic regression.

Next topic: TensorFlow and Keras foundations

Day 2 will translate these manual calculations into Keras layers and tensors while preserving the same conceptual flow.